Mastering The LeetCode

Framework 7 kategori + peta pattern inti + cara membaca soal sampai jadi solusi

Dokumen ini menyusun ulang materi master LeetCode complete dan framework 7 kategori. Bukan sekadar menggabungkan file lama, tapi membentuk satu alur: memahami soal, membangun mesin algoritma, lalu mengenali pattern.

Core idea	Makna
LeetCode = transformasi input ke output	Input masuk, algoritma mengubahnya menjadi output yang benar dan efisien.
Framework 7 kategori	Cara mengisi komponen solusi secara sistematis.
Pattern	Kombinasi khas dari input shape, state, movement, working structure, decision rule, complexity, dan edge case.

Daftar Isi

1. Mental Model: Algoritma = Transformasi Input ke Output

Cara paling dasar melihat soal LeetCode:

```
input -> proses algoritma -> output
```

Prosesnya dikendalikan oleh state, movement, working structure, decision rule, complexity, dan edge case. Jadi saat bingung pattern, jangan mulai dari nebak BFS/DP. Mulai dari membedah komponen mesinnya.

> Pertanyaan utama: input-nya bentuk apa, state-nya apa, geraknya gimana, alat bantunya apa, dan aturan if/else-nya apa?

Contoh: Shortest Path in Binary Matrix

Komponen	Isi
Input shape	Grid 0/1
State	(row, col, distance)
Movement	Gerak 8 arah
Working structure	Queue + visited
Decision rule	Keluar grid skip, cell 1 skip, visited skip, goal return distance
Complexity	O(n^2) time, O(n^2) space
Edge case	Start blocked, goal blocked, grid 1x1, tidak ada path

2. Framework 7 Kategori

Kategori	Pertanyaan inti	Contoh
Input shape	Soal ngasih data bentuk apa?	Array, string, grid, tree, graph, linked list, choices, sorted space
State	Variable/kondisi apa yang berubah?	left/right, row/col, node, path, sum, distance, low/high/mid
Movement	State berubah gimana?	i++, 4 arah, child, choose/undo, expand/shrink, halve
Working structure	Alat internal apa yang dibuat?	Map, Set, Queue, Stack, Heap, visited, path, result, DP table, Trie
Decision rule	If/else untuk skip/lanjut/update/stop?	blocked skip, visited skip, path complete save, can(mid) true shrink
Complexity	Biaya langkah dan memori naik kayak apa?	O(n), O(log n), O(n^2), O(V+E), O(2^n)
Edge case	Kondisi pinggir apa yang bisa merusak?	empty, null, duplicate, no answer, blocked, cycle, overflow, off-by-one

Formula baca soal:

```
Soal mentah
  -> Input shape
  -> State
  -> Movement
  -> Working structure
  -> Decision rule
  -> Complexity
  -> Edge case
  -> Pattern mulai kelihatan
```

3. Isi Core Tiap Kategori

Ini adalah menu mental. Saat lihat soal, lu pilih yang cocok dari tiap kategori dan buang yang tidak relevan.

Input shape

Core choices	Core choices
- Array/List - String - Matrix/Grid - Tree - Graph - Linked List	- Intervals - Sorted array / sorted search space - List of choices - Stream / data datang terus - Number / math input

State

Core choices	Core choices
- index / i - left, right - row, col - current node - current word - level / depth - distance / steps	- path sementara - current sum - window content - visited status - parent / previous - low, high, mid - dp index/state

Movement

Core choices	Core choices
- i++ - left++ / right-- - expand right - shrink left - go to neighbor - go left/right child - go next linked node	- choose item - undo item - split half - relax edge - merge group - move to next DP state

Working structure

Core choices	Core choices
- Set - Map / HashMap - Queue - Stack - Heap / PriorityQueue - Deque - Visited set/matrix - Frequency map	- Parent map - Path list - Result list - DP table - Prefix sum array - Trie - Union Find - Graph adjacency list

Decision rule

Core choices	Core choices
- out of bounds -> skip - blocked -> skip - visited -> skip - duplicate -> skip - invalid window -> shrink - valid window -> update answer	- current == target -> return - path complete -> save result - sum > target -> prune - nums[mid] < target -> move left - can(mid) true -> shrink answer space - roots equal -> already connected

Complexity

Core choices	Core choices
- O(1) - O(log n) - O(n) - O(n log n) - O(n^2)	- O(m*n) - O(V+E) - O(2^n) - O(n!)

Edge case

Core choices	Core choices
- input kosong - input 1 elemen - null root / empty tree - grid kosong - start/goal blocked - tidak ada jawaban - jawaban di awal/akhir - duplicate	- semua elemen sama - angka negatif / 0 - overflow - graph disconnected - cycle - queue/stack/heap kosong - path kosong - off-by-one

4. Pattern Map: Pattern sebagai Kombinasi 7 Kategori

Pattern bukan keyword. Pattern adalah kombinasi yang sering muncul berulang.

Pattern	Input	State	Movement	Working structure	Natural use
BFS	Graph/Grid/Tree	node/cell + distance/level	neighbor/child per layer	Queue + visited	shortest unweighted, level order, spread
DFS	Tree/Graph/Grid	current node/cell	masuk sedalam mungkin	Recursion/Stack + visited	traversal, connected component, path exists
Backtracking	Choices/Grid/String	level + path	choose -> explore -> undo	path + result + used	subsets, permutations, combinations, N-Queens
Binary Search	Sorted/search space	low/high/mid	halve search space	boundary variables	target, lower bound, search on answer
Sliding Window	Array/String	left/right + window	expand/shrink	Map/Set frequency	substring/subarray with condition
Two Pointers	Array/String sorted/linear	left/right	move one/both pointers	pointers only / maybe set	pair sum, palindrome, remove duplicates
Prefix Sum	Array/Grid	running sum/index	accumulate	prefix array/map	range sum, subarray sum
Heap	Stream/List/Graph	top priority item	push/pop priority	Heap/PriorityQueue	top K, scheduler, Dijkstra
DP	Array/String/Grid/Choices	dp state	derive from smaller state	dp table/memo	overlap subproblem, optimize count/min/max
Union Find	Graph edges/groups	parent/root	union/find	parent array + rank	connected components, cycle, grouping
Trie	List of words	current prefix node	char by char	TrieNode children map	prefix search, autocomplete, word dictionary

5. Pattern Cards Inti

BFS - menyebar melebar dulu

Makna	Jawaban naturalnya paling dekat, paling pendek, atau penyebaran per layer.
Working structure	Queue + visited
Contoh	Shortest path unweighted, Rotting Oranges, Word Ladder, Level Order Tree.

DFS - menyelam dalam dulu

Makna	Telusuri satu cabang sampai habis, atau eksplor satu komponen.
Working structure	Recursion/Stack + visited
Contoh	Tree traversal, Number of Islands, path exists, nested structure.

Backtracking - pilih, lanjut, undo

Makna	Problem adalah ruang kemungkinan dan perlu mencoba pilihan.
Working structure	path + result + used/set
Contoh	Subsets, permutations, combinations, N-Queens, Word Search.

Binary Search - mencari boundary

Makna	Ruang jawaban terurut atau ada can(x) monotonic.
Working structure	low/high/mid
Contoh	Search target, lower bound, capacity/speed minimum.

Sliding Window - area aktif bergerak

Makna	Array/string contiguous dengan kondisi window.
Working structure	left/right + freq map/set
Contoh	Longest substring, minimum window, max sum size k.

Two Pointers - dua posisi aktif

Makna	Dua posisi bisa mengurangi pencarian.
Working structure	left/right
Contoh	Sorted two sum, palindrome, remove duplicates, container water.

Heap - prioritas

Makna	Perlu ambil item paling kecil/besar/urgent berkali-kali.
Working structure	PriorityQueue/Heap
Contoh	Top K, task scheduler, Dijkstra, merge K sorted lists.

Trie - prefix tree

Makna	Kata/prefix jadi inti problem.
Working structure	TrieNode children + isWord
Contoh	Autocomplete, search suggestions, Word Search II.

Union Find - group

Makna	Operasi utamanya gabung kelompok dan cek satu grup.
Working structure	parent[] + rank[]
Contoh	Connected components, redundant connection, accounts merge.

DP - simpan subproblem

Makna	Subproblem berulang dan state bisa dibangun dari state kecil.
Working structure	memo / dp table
Contoh	Climbing stairs, coin change, knapsack, LIS, edit distance.

6. Studi Kasus: Cara Mengisi 7 Kategori

Rotting Oranges

Kategori	Isi
Input shape	Grid 0/1/2
State	row, col, minute/layer
Movement	4 arah
Working structure	Queue + fresh counter
Decision rule	Fresh neighbor -> busukkan, fresh--, queue.add. Kosong/keluar/sudah busuk -> skip
Complexity	O(rows*cols)
Edge case	Tidak ada fresh, fresh terisolasi, grid kosong

Word Ladder

Kategori	Isi
Input shape	beginWord, endWord, wordList. Dimodelkan sebagai implicit graph
State	currentWord, level
Movement	ubah 1 huruf jadi kata valid
Working structure	Queue + Set dictionary + visited
Decision rule	word tidak valid/visited -> skip. endWord ketemu -> return level
Complexity	tergantung jumlah kata N dan panjang kata L
Edge case	endWord tidak ada, begin==end, wordList kosong

Backtracking Menu

Kategori	Isi
Input shape	List of lists: makanan, minuman, dessert
State	level + path
Movement	pilih item dari options[level], lalu level+1
Working structure	path + result
Decision rule	level selesai -> copy path ke result. Setelah explore -> undo
Complexity	perkalian jumlah pilihan tiap kategori
Edge case	kategori kosong, options kosong, path kosong valid/tidak

Binary Search on Answer

Kategori	Isi
Input shape	Ruang jawaban angka yang terurut secara valid/tidak valid
State	low, high, mid
Movement	tes mid, buang setengah ruang
Working structure	boundary variables
Decision rule	can(mid) true -> cari lebih kecil; false -> cari lebih besar
Complexity	O(log range * cost can)

Kategori	Isi
Edge case	boundary salah, infinite loop, can tidak monotonic

Heap Scheduler

Kategori	Isi
Input shape	Jobs datang terus: id, priority/scheduledAt, label
State	top job / heap size
Movement	push job baru, pop job paling prioritas
Working structure	Heap/PriorityQueue
Decision rule	Less(): priority terkecil keluar dulu. Kalau due -> process
Complexity	push/pop O(log n), peek O(1)
Edge case	heap kosong, priority sama, server restart kalau in-memory

Trie Search Suggestion

Kategori	Isi
Input shape	List of words + query prefix
State	current TrieNode + prefix index
Movement	turun char by char
Working structure	TrieNode children Map + isWord
Decision rule	char tidak ada -> prefix tidak ditemukan. isWord true -> kata valid
Complexity	O(length of word/prefix) per search
Edge case	prefix kosong, kata duplicate, huruf besar/kecil

7. Tree Family dalam Programming

Tree adalah konsep parent-child. Banyak struktur data memakai ide tree, tapi aturan dan implementasinya beda.

Jenis	Node/implementasi	Aturan khusus	Dipakai untuk
General Tree	value + List children	child bisa banyak	folder, kategori, menu, comment, widget tree
Binary Tree	value + left + right	maksimal 2 anak	tree traversal, recursion
BST	left/right	left < root < right	search/insert terurut, kth smallest
Heap	List item + rumus index	parent lebih prioritas dari child	priority queue, top K, Dijkstra
Trie	children Map + isWord	path membentuk prefix/kata	autocomplete, dictionary, prefix search
Segment Tree	array tree rentang	node simpan info range	range query/update
AST	node kode	struktur sintaks kode	compiler, linter, formatter
Widget Tree	Widget sebagai node	aturan child sesuai widget	Flutter UI

Catatan penting:

- Backtracking tree sering cuma visualisasi ruang kemungkinan, bukan input asli.

- Heap secara konsep tree, tapi implementasi biasanya List/Array.

- Trie adalah tree khusus. Semua Trie adalah tree, tapi tidak semua tree adalah Trie.

- Flutter Widget Tree adalah general tree secara struktur, tapi domain-specific karena tiap widget punya aturan child sendiri.

8. Template Dart Minimal yang Sering Kepakai

Bukan untuk dihafal mentah. Pakai sebagai bentuk mesin dasar.

BFS Grid

```
import 'dart:collection';

final dirs = [[1,0],[-1,0],[0,1],[0,-1]];

int bfs(List<List<int>> grid) {
  final rows = grid.length;
  final cols = grid[0].length;
  final q = Queue<List<int>>();
  final visited = List.generate(rows, (_) => List.filled(cols, false));

  q.add([0, 0, 0]); // row, col, distance
  visited[0][0] = true;

  while (q.isNotEmpty) {
    final cur = q.removeFirst();
    final r = cur[0], c = cur[1], d = cur[2];
    for (final dir in dirs) {
      final nr = r + dir[0], nc = c + dir[1];
      if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
      if (grid[nr][nc] == 1 || visited[nr][nc]) continue;
      visited[nr][nc] = true;
      q.add([nr, nc, d + 1]);
    }
  }
  return -1;
}
```

Backtracking List Kategori

```
void backtrack(
  List<List<String>> options,
  int level,
  List<String> path,
  List<List<String>> result,
) {
  if (level == options.length) {
    result.add(List.from(path));
    return;
  }

  for (final choice in options[level]) {
    path.add(choice);                  // choose
    backtrack(options, level + 1, path, result); // explore
    path.removeLast();                 // undo
  }
}
```

Binary Search Boundary

```
int binarySearchBoundary(int low, int high, bool Function(int) can) {
  while (low < high) {
    final mid = low + ((high - low) ~/ 2);
    if (can(mid)) {
      high = mid;
    } else {
      low = mid + 1;
    }
  }
  return low;
}
```

9. Cara Latihan: Dari Soal Mentah ke Solusi

Setiap kali ketemu soal, isi template ini sebelum lihat editorial.

Langkah	Pertanyaan
1. Input shape	Soal ngasih data bentuk apa?
2. Goal	Output yang diminta apa? count, bool, path, max/min, semua kemungkinan?
3. State	Variable kondisi apa yang berubah selama proses?
4. Movement	State pindah/berubah gimana?
5. Working structure	Butuh alat internal apa? Queue, Map, Stack, Heap, path, dp?
6. Decision rule	If/else untuk skip, lanjut, update, save, return, undo apa?
7. Complexity	Target time/space yang masuk akal apa?
8. Edge case	Kasus pinggir apa yang bisa bikin salah?

Drill 5 menit untuk satu soal

- Tulis input shape dan output dalam 1 kalimat.
- Tulis state utama. Kalau tidak bisa, berarti algoritma belum kebentuk.
- Tulis movement: dari state sekarang ke state berikutnya gimana.
- Tulis working structure yang dibutuhkan dan alasannya.
- Tulis 3 decision rule paling penting.
- Baru coding.

Kalimat penutup

Mastery bukan menghafal semua tanda soal. Mastery adalah kemampuan membentuk mesin solusi: input dibaca, state bergerak, working structure membantu, decision rule mengontrol, complexity aman, edge case tidak merusak.